AI Print Estimator

What you need to process the data and run this engine

Single-file MIS · **paper_calculator.html** · prepared 24 June 2026

The product is one self-contained HTML file (HTML + CSS + JavaScript) that runs a complete print estimation / imposition MIS — paper requirement, best sheet & machine, colour / plate / impression costs, wastage, binding and a priced quotation. It needs **no server and no internet** to operate. The lists below separate (A) what an *operator* needs to run it, (B) what a *maintainer* needs to edit / verify / publish it, (C) the data the engine carries, (D) the inputs you give per job, and (E) what is still pending to make every rate fully real.

A. To RUN the estimator (operator)

Requirement	Detail
Web browser	Any modern browser — Chrome, Edge or Firefox. Nothing else to install.
The app file	**Paper_Calculator.html** — double-click to open. Works fully offline; copies kept on Desktop and Downloads.
OR the live link	https://mounojitd.github.io/mounojitd-ai-print-estimator/ — open in a browser, no login. Updates appear ~1 min after a publish; just refresh.
Internet	Not required for the offline file. Required only to open the live link.
Account / setup	None. No install, no sign-in, no configuration.

B. To EDIT, VERIFY & PUBLISH (maintainer / developer)

Tool	Why it is needed
Python 3.12	Import NKK sir's Excel / PDF data into the rate masters and render his hand-drawn PDFs. Path: C:\Users\AR04\AppData\Local\Programs\Python\Python312\python.exe
Python libraries	openpyxl (read Excel masters), pypdf & PyMuPDF/fitz (read & render his PDF drawings), reportlab (generate documents like this one).
Node.js	Verify before publishing: syntax-check the script (`node --check`) and run math harnesses that re-prove sir's hand numbers (1,105 / 6,000 / 0.0929 etc.) before going live.
Git + GitHub	Repo **Mounojitd/mounojitd-ai-print-estimator** (branch main); credentials cached. Push = publish to GitHub Pages.
publish.ps1	One-step publish: copy paper_calculator.html -> index.html + backend/static + Desktop + Downloads, then commit & push. Pages rebuilds in ~1 min.
Text editor	Any editor to change the single HTML file. No build step, no framework, no dependencies to compile.

C. Data the engine already carries (embedded rate databases)

These are baked into the HTML / repo so the estimate runs offline. All sourced from NKK sir's real sheets.

Database	Contents
PAPER_DB	457 paper rows (type / grade / GSM / sheet size / Rs-per-sheet). Drives auto paper, cover and now endpaper pricing.
MACHINES	8 offset presses + 4 digital, each with max paper / max print / gripper / colours / speed (rebuilt from sir's Process master).
Printing rate cards	Per-machine lot + per-1000 tier + plate / colour rates (Anderson / Naba Mudran agreement).
Post-press master	Binding by pages & quantity, lamination / UV / varnish / aqueous coating, embellishments (Spot UV / foil / emboss), casing-in.

Database	Contents
Vendor CSVs	db/files/vendor_db/*.csv — the raw masters (paper pricing, printing, post-press, material, transport).
Source of truth	MASTER_PROJECT_STATE.md — full decision log + canonical hand numbers; read first when resuming.

D. Inputs you give per job (to produce an estimate)

Group	What you enter
Product & size	Product type (book / catalogue / leaflet / poster / calendar / card…), finished size, orientation, page count, quantity.
Text paper	Paper type + grade + GSM (engine auto-prices from PAPER_DB), or a manual Rs/sheet.
Colours & plates	Front + back colours (presets 1+0 … 6+6); plates and impressions are derived automatically.
Coating	Inside / text coating choice and whether it applies to all sheets or cover-only.
Cover (optional)	Separate cover on/off; cover GSM, colours, coating, up to 3 embellishments; binding type (auto-recommended by spine).
Margins (mm)	Bleed, gripper, side-lay, backside, paper-trim — used for imposition and the production-size diagram.
Hard case (optional)	Board thickness, square / overhang, joint, turn-in, case material (PLC / rexine / cloth / velvet), endpaper type+GSM, casing-in, headband / ribbon / jacket.
Commercial	Margin %, optional overhead %, optional GST % (added manually per sir).

E. Still pending from NKK sir (to make every rate fully real)

The engine runs today with these as editable defaults; the numbers below are the only placeholders left.

Item	What is needed
Hard-case materials	Greyboard Rs/sheet + sheet size by thickness; rexine / cloth / velvet Rs/sq.in or Rs/m; headband / ribbon / dust-jacket Rs/book. (Endpaper is now auto-priced from the paper DB — done.)
Fold forms 24 / 32pp	Hand-drawn page-order forms so 24pp and 32pp use a real fold sequence (4 / 8 / 12 / 16pp already done).
Digital click rate	Rs-per-click card for digital presses (digital currently borrows the offset rate).
Grain direction	The rule for grain-direction warnings / locking.
Wastage brackets	The exact 15% / 25% (and 5%) rule — which quantity / run falls in which bracket.
Binding grid	A fuller binding grid or formula (size + forms + paper + quantity + lot-minimum) to replace the current sparse sample points.

Bottom line: to **run** it you need only a browser and the HTML file. To **maintain & publish** it you need Python 3.12, Node.js and Git. The engine already carries all real rate data except the few hard-case material rates and refinement items listed in section E.